Training an LLM to Reason Using Tunix Supported RL

CJ Jones and Adam Stein

1. Introduction

Large language models (LLMs) have rapidly advanced in their ability to answer complex questions, yet they often provide only a final answer without revealing the reasoning process that produced it. This lack of transparency makes it difficult to evaluate model reliability, diagnose sources of error, or build trust in deployed systems. Recent work has shown that generating explicit reasoning traces—whether as chain-of-thought explanations or structured intermediate steps—can improve accuracy, interpretability, and alignment. However, training models to consistently produce high-quality reasoning traces remains a challenging problem. It requires not only curated data but also post-training techniques capable of shaping global output behaviors rather than merely optimizing token-level predictions.

This project addresses that challenge by using Tunix, Google's new JAX-native library for LLM post-training, to teach an open-weight Gemma model to reason transparently. Leveraging Tunix's reinforcement learning (RL) capabilities, the model is fine-tuned to produce structured reasoning sequences prior to generating its final answer. Unlike conventional supervised fine-tuning, which encourages imitation of labeled examples, reinforcement learning allows the model to be optimized against a reward function that evaluates complete outputs for correctness, coherence, formatting compliance, and reasoning quality. This enables the system to develop more robust multi-step reasoning behaviors that extend beyond simple pattern matching.

The aim of this work is to construct a training pipeline that demonstrates how RL can be used to shape reasoning-oriented behaviors in open-source LLMs. The pipeline integrates Gemma, Tunix, TPU compute, and a custom reward function that scores both reasoning traces and final answers. The resulting model is designed not only to produce correct solutions, but also to articulate how those solutions were derived. In doing so, the project illustrates how reinforcement-based post-training can enhance the interpretability, trustworthiness, and practical usability of open-weight language models.

2. Literature review

2.1 DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via RL

A central reference point for this project is the paper by DeepSeek-AI et al. (2025), which first applied large-scale reinforcement learning to improve LLM reasoning without relying heavily on supervised fine-tuning. DeepSeek-R1-Zero in particular serves as an early demonstration that a base language model can acquire strong chain-of-thought behavior purely through RL, using Group Relative Policy Optimization (GRPO) as the training algorithm. In this setup, the model is treated as a policy $\pi\theta(o|q)$ that maps questions *q* to full output sequences *o*and learning proceeds by sampling groups of candidate completions for each question and scoring them with simple, rule-based rewards.

GRPO modifies the standard PPO-style objective by replacing the usual learned value function with a group-based baseline. For each question, *G* candidate outputs $\{o_1,\ldots,o_G\}$, are sampled from the old policy $\pi_{\theta old}$, and each completion receives a scalar reward r_i. Advantages are then computed by normalizing within the group, so that the best completions within a group get positive advantage and the worst get negative advantage, without requiring a separate critic network. Conceptually, the GRPO with its KL based objective function helps to directly optimize for better full answers rather than better token-by-token predictions, while keeping the updated model close to a safe reference.

In DeepSeek-R1-Zero and DeepSeek-R1, this algorithm is coupled with a simple rule-based reward model. This includes an accuracy reward that checks whether the final answer matches a ground-truth solution, and format reward that enforces the policy that all reasoning processes appear between dedicated tags, such as <think> </think>, and the final answer appears in a dedicated answer tag. Importantly, the reward is defined on the entire output trajectory and can incorporate structural constraints in addition to correctness. This design is mirrored in the present project in which the Gemma-based system uses GRPO with group sampling over multiple completions per GSM8K question, and the custom reward functions similarly decompose into

(i) strict format rewards for <reasoning> and <answer> blocks and
(ii) correctness rewards based on numeric answer extraction.

As a matter of code structure, the GRPO objective, with its group-normalized advantages and KL regularization, explains why the project code is adapted to generate multiple candidate solutions per question, define rewards at the sequence level, and tune KL and clipping parameters so that the trained LoRA adapters improve reasoning while remaining faithful to the original Gemma behavior.

2.2 GRPO Demo – Gemma3-1B by WindMaple

This project also builds directly on prior work demonstrating reinforcement learning with open-weight models using the GRPO algorithm. In particular, the widely circulated Kaggle

notebook “GRPO Demo – Gemma3-1B” by WindMaple provided an early, practical template for applying GRPO within the Tunix ecosystem. That notebook introduced a minimal but functional pipeline for sampling model outputs, computing reward signals, and performing group-relative optimization on the Gemma-3 1B model. It also illustrated the use of LoRA adapters for efficient parameter updates and offered a reference implementation of several format- and correctness-based reward functions originally designed for GSM8K-style tasks. As such, the Kaggle demo served as an essential foundation by demonstrating how GRPO can be operationalized on TPU hardware through Tunix’s JAX-native abstractions.

A goal of this project is to extend and generalize this baseline implementation. The primary addition is a training procedure that was modularized into a two-stage Planner Solver architecture, which is intended to allow a high-level task decomposition and detailed reasoning generation to be optimized separately. This contrasts with the single-model design of the original demo. Additionally, the reward functions were revised to enforce stricter reasoning structure, support approximate formatting, and incorporate numerically graded correctness rather than binary scoring. Finally, a full evaluation framework was added, enabling consistent measurement of exact accuracy, partial correctness, and structural compliance across large test sets.

3. Dataset and Test Train Split

This project employs the GSM8K dataset, a widely used benchmark for grade-school mathematics reasoning. GSM8K consists of 8,500 high-quality, human-written word problems paired with free-form natural language rationales and final numeric answers. The dataset is specifically designed to evaluate multi-step arithmetic reasoning, making it well suited for studying structured reasoning behavior in language models.

The dataset contains two predefined subsets: a training set of 7,473 examples and a held-out test set of 1,319 examples. These official splits are preserved in order to maintain comparability with prior work. Each example contains a problem statement and an answer expressed in the final number format. During preprocessing, the final numeric answer is extracted and normalized, while the question text is retained in its original form.

To support efficient TPU execution through Tunix, the data is streamed using `grain.MapDataset` and batched with small micro-batches that match the RL rollout structure. Shuffling is applied only to the training portion of the dataset, while the test set is kept in fixed order for reproducibility. This separation ensures that no test data influences model optimization, enabling clean measurement of improvements induced by reinforcement learning and the two-stage planner–solver architecture.

4. Methodology

This project implements a two-stage reinforcement learning framework designed to shape structured reasoning in a Gemma3 1B language model. Instead of relying on supervised fine-tuning of base parameters, the approach uses lightweight LoRA adapters and Google Tunix's GRPO algorithm to optimize the model's behavior at the level of whole generated sequences. The resulting system is composed of three major components: initialization of LoRA-augmented Gemma models, a modular Planner–Solver architecture, and a reinforcement learning procedure driven by structured reward functions.

4.1 Model Initialization and LoRA Adapter Construction

The process begins with the Gemma-3 1B Instruct checkpoint, which is loaded into a JAX-native environment to take advantage of TPU acceleration within Tunix. The Gemma backbone remains entirely frozen throughout training. Instead, LoRA adapters are inserted into key projection and attention modules using qwix.apply_lora_to_model, allowing a small set of trainable parameters to govern all learning behavior. Separate LoRA parameter sets are created for three models—the baseline, the planner, and the solver—ensuring that each model can acquire distinct functional roles without modifying the underlying Gemma weights. This modularity enables rapid experimentation, reproducible behavior, and a clear separation between high-level planning and detailed reasoning.

4.2. Planner Solver Architecture

A two-model architecture is adopted to decompose the reasoning process into planning and execution. This design draws inspiration from recent work in structured reasoning systems, where the decomposition of tasks has been shown to improve both controllability and interpretability.

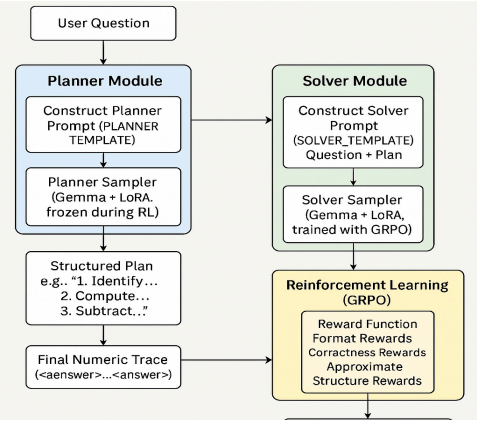


Fig 1. Visual Diagram of Two Model Planner Solver Pipeline

4.3 Planner Model

The Planner model receives the original question and produces a short, numbered plan wrapped in <plan>...</plan> tags. It is trained exclusively to produce syntactically correct and consistently structured plans. During reinforcement learning, the Planner remains frozen; only its LoRA weights learned earlier are used. This stability ensures that the Solver encounters a steady distribution of plans and can optimize its reasoning behavior without absorbing fluctuations from an evolving planning policy.

4.4 Solver Model

The Solver model receives both the question and the planner-generated plan, which are combined into a strict prompt template that includes <reasoning> and <answer> blocks. The template requires the Solver to follow the steps outlined in the plan, produce a coherent reasoning trace, and conclude with a single numeric answer. Only the Solver's LoRA parameters are updated during reinforcement learning. By isolating RL updates to the Solver, the system cleanly separates task decomposition from reasoning execution, allowing each component to specialize without interference.

4.5 Reinforcement Learning with GRPO

Reinforcement learning is conducted using Tunix's implementation of GRPO. Unlike PPO-based RLHF pipelines, GRPO evaluates groups of completions per prompt and normalizes their rewards, thereby stabilizing training and eliminating the need for a learned value function. For each input question, multiple solver outputs are generated. Their rewards are computed, and GRPO assigns each output a relative advantage based on its deviation from the group mean. The objective is a sequence-level variant of clipped policy improvement where (A_i) represents the normalized advantage of each output and (\beta) controls the strength of KL regularization against the reference model.

This method is particularly well suited to reasoning tasks because it evaluates whole sequences rather than token-level predictions, allowing reward functions to target global properties such as logical coherence, tag structure, or numeric correctness.

4.6 Reward Function Design

A composite reward function is used during Solver training, combining structural and semantic criteria. The reward consists of three components:

A strict regular expression checks whether the output conforms to the full template containing <reasoning> and <answer> blocks. A reward of +3.0 is assigned to perfectly matched structures. This component enforces reliable formatting, which is essential for automated parsing and evaluation.

A softer score evaluates the presence of properly matched tags. Points are added for correctly placed start and end tags, while penalties are applied for missing or duplicated tags. This design prevents the model from receiving zero reward for minor formatting deviations, guiding it gradually back toward the expected structure.

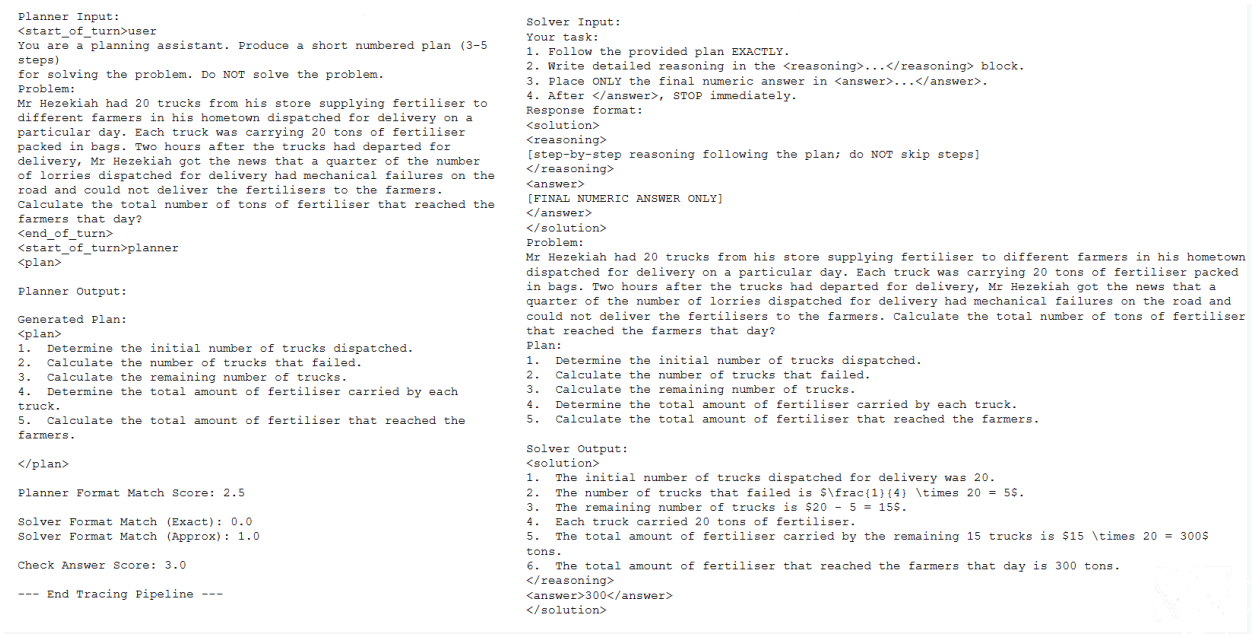
The final numeric answer is extracted from the <answer> block and compared with the ground truth. Exact numeric matches receive the highest reward, while partial credit is awarded for predictions falling within ±10% or ±20% of the correct value. Penalties are applied for non-numeric outputs, parsing failures, or clearly incorrect predictions. This component constitutes the primary signal driving improvement in mathematical accuracy.

4.7 Training Workflow

The full training workflow proceeds through several coordinated stages that collectively transform the base Gemma model into a structured reasoning system. The process begins with initialization of the Gemma-3 1B model and the attachment of LoRA adapters, enabling efficient, low-overhead fine-tuning while preserving the integrity of the pretrained backbone. This configuration ensures that all subsequent optimization remains lightweight, stable, and easily comparable to the original model.

Training then divides into two complementary branches: the Planner and the Solver. The Planner is trained first, using its own GRPO gradient descent policy and structurally oriented rewards that emphasize the production of concise, well-organized plans. These rewards focus entirely on format, such as the presence of proper tags and a reasonable sequence of steps, while deliberately excluding correctness. Once this behavior stabilizes, the Planner is frozen so that its outputs remain fixed throughout the remainder of the pipeline.

The Solver is then trained using reinforcement learning with the GRPO algorithm. In this stage, the Solver receives both the original question and the structured plan produced by the frozen Planner. GRPO generates multiple candidate trajectories, computes advantages relative to a group baseline, and updates the model toward behaviors that yield higher composite rewards. A KL-penalty term prevents excessive drift from the pretrained distribution, promoting stable updates.



```
Planner Input:
<start_of_turn>user
You are a planning assistant. Produce a short numbered plan (3-5
steps)
for solving the problem. Do NOT solve the problem.
Problem:
Mr Hezekiah had 20 trucks from his store supplying fertiliser to
different farmers in his hometown dispatched for delivery on a
particular day. Each truck was carrying 20 tons of fertiliser
packed in bags. Two hours after the trucks had departed for
delivery, Mr Hezekiah got the news that a quarter of the number
of lorries dispatched for delivery had mechanical failures on the
road and could not deliver the fertilisers to the farmers.
Calculate the total number of tons of fertiliser that reached the
farmers that day?
<end_of_turn>
<start_of_turn>planner
<plan>

Planner Output:

Generated Plan:
<plan>
1.  Determine the initial number of trucks dispatched.
2.  Calculate the number of trucks that failed.
3.  Calculate the remaining number of trucks.
4.  Determine the total amount of fertiliser carried by each
truck.
5.  Calculate the total amount of fertiliser that reached the
farmers.

</plan>

Planner Format Match Score: 2.5

Solver Format Match (Exact): 0.0
Solver Format Match (Approx): 1.0

Check Answer Score: 3.0

--- End Tracing Pipeline ---
```

```
Solver Input:
Your task:
1. Follow the provided plan EXACTLY.
2. Write detailed reasoning in the <reasoning>...</reasoning> block.
3. Place ONLY the final numeric answer in <answer>...</answer>.
4. After </answer>, STOP immediately.
Response format:
<solution>
<reasoning>
[step-by-step reasoning following the plan; do NOT skip steps]
</reasoning>
<answer>
[FINAL NUMERIC ANSWER ONLY]
</answer>
</solution>
Problem:
Mr Hezekiah had 20 trucks from his store supplying fertiliser to different farmers in his hometown
dispatched for delivery on a particular day. Each truck was carrying 20 tons of fertiliser packed
in bags. Two hours after the trucks had departed for delivery, Mr Hezekiah got the news that a
quarter of the number of lorries dispatched for delivery had mechanical failures on the road and
could not deliver the fertilisers to the farmers. Calculate the total number of tons of fertiliser
that reached the farmers that day?
Plan:
1.  Determine the initial number of trucks dispatched.
2.  Calculate the number of trucks that failed.
3.  Calculate the remaining number of trucks.
4.  Determine the total amount of fertiliser carried by each truck.
5.  Calculate the total amount of fertiliser that reached the farmers.

Solver Output:
<solution>
1.  The initial number of trucks dispatched for delivery was 20.
2.  The number of trucks that failed is $\frac{1}{4} \times 20 = 5$.
3.  The remaining number of trucks is $20 - 5 = 15$.
4.  Each truck carried 20 tons of fertiliser.
5.  The total amount of fertiliser carried by the remaining 15 trucks is $15 \times 20 = 300$
tons.
6.  The total amount of fertiliser that reached the farmers that day is 300 tons.
</reasoning>
<answer>300</answer>
</solution>
```

Fig 2. Example of Two Step Pipeline Evaluated via Custom Rewards Function

After optimization, the trained Planner and Solver are recombined into a unified inference-time pipeline. Evaluation then measures exact and approximate correctness as well as structural reliability. Through this staged design, the system acquires not only accuracy but also the ability to articulate interpretable, well-structured reasoning traces.

5. Results

The empirical findings from this project are presented in two phases. The first phase reproduces the GRPO demonstration notebook from WindMaple as a single-stage baseline experiment, allowing direct comparison with an established reference implementation. The second phase evaluates the full two-stage Planner–Solver architecture developed in this work, which extends the single-model setup into a modular reasoning system. Together, these experiments illustrate both the strengths of conventional GRPO post-training and the additional benefits conferred by structured architectural design.

5.1 Demo Training (Single-Stage Baseline Performance)

The initial experiment follows the structure of the WindMaple GRPO demo, using a single Gemma model to generate both reasoning traces and final answers. A subset of 100 GSM8K-style math problems was used for training, with a separate 100-question split held out for evaluation. The reward curves associated with this run are shown in the red graphs.

During training, the reward signals behaved markedly differently depending on their complexity. The check_answer reward, which reflects exact numerical correctness, remained highly volatile

throughout the run. It frequently oscillated between negative and positive values without displaying a clear upward trajectory. Such instability is characteristic of sparse mathematical correctness signals: small deviations in any part of the chain-of-thought can invalidate an entire solution, and the model's need to explore diverse reasoning paths injects even more variance.

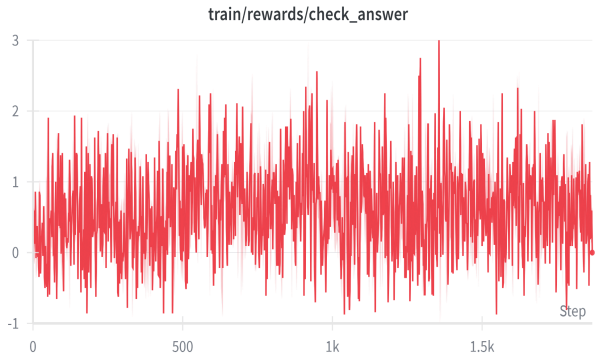


Fig 3. Demo Check Answer Training Rewards

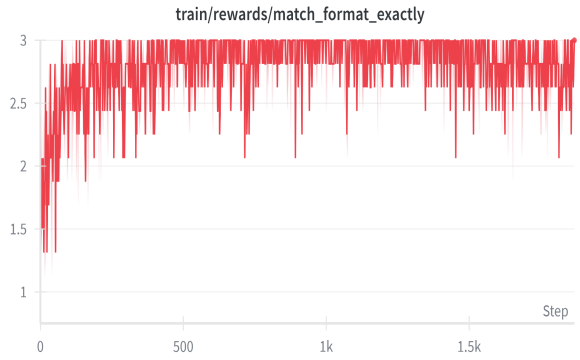


Fig 4. Demo Match Format Exactly Reward

Nonetheless, the signal did not deteriorate or collapse, indicating that GRPO maintained stable and non-destructive learning dynamics.

The two formatting-related rewards behaved much more predictably. The match_format_approximately reward climbed rapidly in the early stages of training and settled close to its maximum, indicating that the model quickly internalized the broad structure required by the reasoning and answer tags. The match_format_exactly reward followed the same general pattern but exhibited slower and more irregular convergence, reflecting the greater difficulty of adhering to strict token-level formatting constraints. Even with minor fluctuations, the model ultimately produced highly consistent, template-aligned outputs.

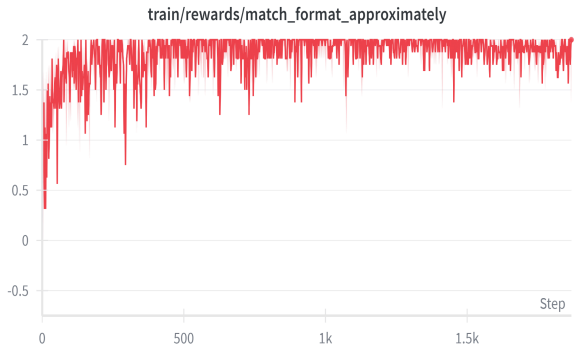


Fig 5. Demo Match Format Approx Rewards

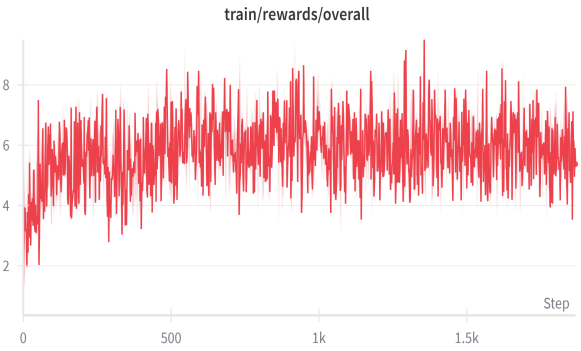


Fig 6. Demo Overall Training Rewards

Evaluation of the single-stage model before and after training highlights the effectiveness of GRPO in shaping both formatting and correctness behavior:

Pre GRPO Training			**Post GRPO Training**	
Exact accuracy:	14.5%		Exact accuracy:	46.25%
Partial accuracy:	16.0%		Partial accuracy:	49.0%
Format accuracy:	46.25%		Format accuracy:	96.25%

Table 1. Demo Pre and Post Training Evaluation on 100 Hold Out Test Math Questions

This dramatic improvement, particularly in structural reliability, underscores the ability of GRPO to enforce global output patterns and encourage more stable reasoning behaviors even in compact training settings.

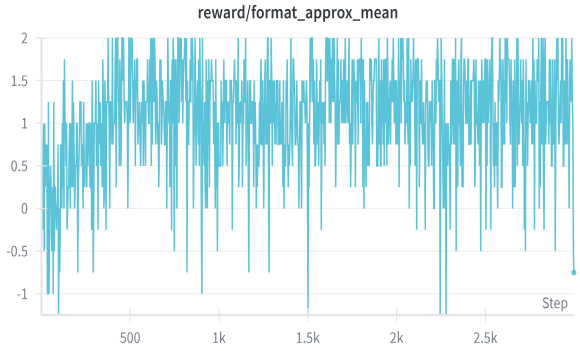
5.2 Two-Stage Planner–Solver Results

The second experimental phase evaluates the full two-stage system developed in this project. Here, a frozen Planner generates high-level task decompositions, while a Solver—initialized from the same pretrained Gemma model but equipped with separate LoRA adapters—is optimized through GRPO using both the problem prompt and the Planner's output. This division of labor is intended to reduce exploration complexity for the Solver and promote more structured reasoning.

Before reinforcement learning, the Solver operates solely under supervised fine-tuning, and its outputs exhibit substantial inconsistency in both formatting and reasoning alignment. Evaluation prior to GRPO reflects these limitations clearly.
Once GRPO is introduced, the Solver begins to respond much more reliably to the structural cues provided by the Planner. Because the Planner remains frozen throughout reinforcement learning, its outputs act as a stable scaffold that guides the Solver toward coherent reasoning traces. The RL stage therefore focuses exclusively on refining detail-level reasoning, improving formatting consistency, and increasing the likelihood of correct intermediate and final answers.

The two format-related reward curves provide a clear picture of how quickly the Solver learned to internalize structural constraints during GRPO training. The approximate-format reward displays substantial early variability, but it steadily trends upward during the first several hundred steps, eventually stabilizing near its theoretical maximum. Even though periodic dips remain visible throughout training, the overall density of high-scoring updates shows that the model consistently absorbed the broader structural expectations even when minor formatting imperfections were present. The exact format reward exhibits an even more dramatic form of stabilization where, after a brief warm-up period, the Solver begins to produce strictly correct formatting with high regularity. Taken together, these curves demonstrate that the model can

reliably learn rigid syntactic conventions under reinforcement learning and maintain them across thousands of updates.



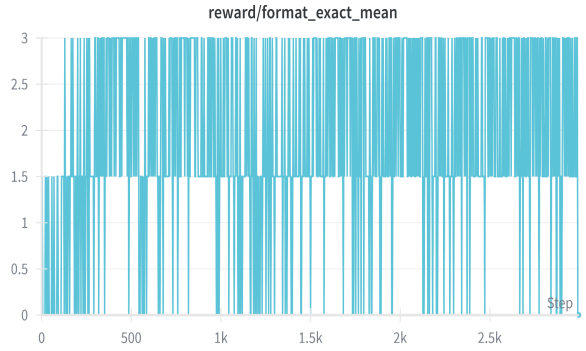


Fig 7. Two Step Format Approx Rewards

Fig 8. Two Step Match Format Exactly Reward

The answer_mean reward shows a more gradual and uneven trajectory, consistent with the difficulty of optimizing semantic correctness through sparse and brittle signals. Early in training, the reward distribution is tightly centered around zero and includes frequent negative excursions, reflecting the model's limited ability to complete multi-step numerical reasoning reliably. Over time, however, the upper envelope of the reward distribution rises noticeably. Despite this improvement, the reward remains substantially noisier than the formatting measures, with a persistent band of negative outcomes reflecting answer errors that occur even when format is perfect. Nonetheless, the upward drift in the reward distribution demonstrates that GRPO successfully nudges the model toward better mathematical accuracy across training.

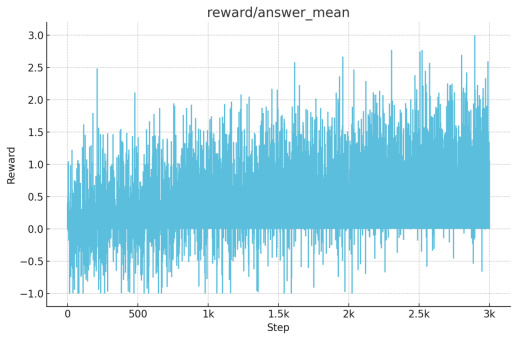


Fig 9. Two Step Check Answer Training Rewards

Post training evaluation demonstrates substantial gains across all metrics:

Pre GRPO Training			**Post GRPO Training**	
Exact accuracy:	31.0%		Exact accuracy:	38.5%
Partial accuracy:	34.5%		Partial accuracy:	42.0%
Format accuracy:	18.5%		Format accuracy:	47.5%

Table 2. Two Step Pipeline Pre and Post Training Evaluation on 100 Hold Out Test Questions

These results reveal several advantages of the two-model approach. Although the total accuracy gains are slightly smaller than those observed in the single-stage demonstration, the two-stage system produces more stable and interpretable reasoning traces, and its formatting reliability improves dramatically despite not being directly fine-tuned for structure. The Planner's high-level guidance appears to constrain the search space in a favorable way, enabling more consistent RL updates and a smoother optimization trajectory.

5.3 Summary of Findings

Taken together, the results confirm the viability of reinforcement learning as a mechanism for shaping reasoning-oriented behavior in compact open-weight language models. GRPO proves especially effective at enforcing structural consistency, while correctness signals remain inherently noisy but still responsive to RL guidance. The two-stage Planner–Solver design offers further benefits by organizing the reasoning process into modular components, enabling more stable chain-of-thought generation and improved generalization on held-out problems. The combined evidence suggests that modular reasoning architectures paired with GRPO represent a promising direction for improving interpretability, structure, and reliability in open-weight LLMs.

6. Discussion

The experimental results offer a valuable perspective on the strengths and limitations of reinforcement-trained reasoning systems, especially when comparing a single-stage GRPO model to the more modular two-stage Planner Solver architecture. One of the most notable findings is that, after adjusting for strict formatting requirements, the single-stage baseline actually outperforms the two-stage pipeline in both exact and partial correctness. This relative underperformance of the two-stage model is not surprising given that it introduces new potential points of failure, most importantly, the transfer of information from a trained Planner to a Solver

that depends on that plan as part of its context. The baseline model, by contrast, benefits from a simpler and more direct optimization landscape.

At the same time, the two-stage pipeline clearly demonstrates significant room for growth. The formatting performance of the Planner Solver model improved after RL, but remained far below the levels reached by the single-stage system. This gap suggests that the formatting portion of the reward signal was not fully propagated across both modules, and that errors introduced in the Planner's output could cascade into degraded Solver outputs. A more robust approach might involve jointly optimizing the Planner and Solver under some shared subset of rewards, or allocating separate but coordinated reward components that explicitly score the Planner's contributions.

Beyond architectural considerations, several practical constraints hindered training efficacy. RAM limitations in particular restricted both prompt size and context windows, occasionally resulting in truncated prompts or incomplete chain-of-thought outputs. These issues also exacerbated the caching-related failures observed during JAX compilation and decoding. Increasing memory would allow longer, more stable prompts to be passed to the Solver, improve sampling stability, and—crucially—reduce the risk of dropped Planner content, which otherwise compromises downstream reasoning consistency.

The most promising direction for improvement lies in designing a more sophisticated training strategy for the Planner itself. In the current approach, the Planner is trained only once and then held fixed during Solver training, meaning its plans never benefit from the correctness signal that the Solver eventually receives during GRPO. A stronger approach would be to introduce an additional training phase, effectively a third GRPO loop, in which a frozen, well-trained Solver evaluates Planner outputs and returns a reward signal based on whether the Solver succeeds when guided by that plan. This inversion of roles would allow correct downstream answers to "flow upward" as a supervisory signal for improving planning quality.

The example output in Figure 10 below illustrates precisely why such a mechanism is needed.

Despite receiving a decent but incomplete plan, the Solver produces incorrect reasoning and an incorrect answer, 8 instead of 32. The solution even contains an internal mistake, 40/2 = 20, before then concluding the answer is 8. This breakdown shows how an insufficient or shallow plan can mislead an otherwise capable Solver and why structural improvements to the Planner would propagate meaningful benefits downstream.

Strategies such as lengthening the plan, enforcing intermediate computation steps, or requiring the Planner to output symbolic constraints could all improve reliability. A jointly trained or reward-sharing Planner–Solver pair would likely produce more coherent multi-step reasoning.

```
=== TRAINED PLANNER ===
1. Define the variables: Let 'x' represent the length of the shorter piece.
2. Express the longer piece in terms of 'x': The longer piece is 4 times longer than
the shorter piece, so its length is 4x.
3. Set up an equation: The total length of the two pieces is 40 feet, so x + 4x = 40.
4. Solve for x: Combine like terms and simplify the equation to find the length of
the shorter piece.


=== TRAINED SOLVER OUTPUT ===
<solution>
x = 40/2 = 20 feet.
<reasoning>
The problem states that the longer piece is 4 times longer than the shorter piece.
We are given that the total length of the two pieces is 40 feet.
Let 'x' be the length of the shorter piece.
Then the length of the longer piece is 4x.
We can set up the equation: x + 4x = 40.
Simplifying, we get 5x = 40. Dividing both sides by 5, we get x = 8
</reasoning>
<answer>8</answer>
```

Fig 10. Example Output of Incorrect Answer of 1 of 100 Hold Out Test Math Questions

Finally, several broader observations emerge from this work. First, the combination of lightweight LoRA adapters and GRPO proved highly effective at shaping structured reasoning behavior even under heavy hardware constraints. Second, the experiments reinforce the idea that modular reasoning architectures offer strong interpretability advantages, even if their initial performance lags behind monolithic models. Third, the results emphasize the necessity of designing reward functions that flow across the entire pipeline rather than terminating at the outputs of the Solver alone. With additional compute, a more expressive reward design, and multi-stage GRPO training, the performance of the Planner Solver pipeline could likely approach or surpass the single-stage baseline while offering clearer, more human-interpretable reasoning traces.

7. References

- https://www.kaggle.com/competitions/google-tunix-hackathon/data
- https://github.com/google/tunix
- https://pypi.org/project/google-tunix/
- https://www.kaggle.com/code/windmaple/grpo-demo-gemma3-1b
- Saha, Swarnadeep, et al. "Learning to plan & reason for evaluation with thinking-llm-as-a-judge." arXiv preprint arXiv:2501.18099 (2025).
- DeepSeek-AI et al., "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning" (2025), which explores using reinforcement learning to enhance the reasoning capabilities of Large Language Models (LLMs).